Server-Client 숫자 야구 게임
:Thread와 Process 차이에 대한 보고서

이 수 아 (Lee Suah)

0. 요약

Server-Client 형태의 TCP 통신을 하는 숫자 야구 게임을 구현하였다. Thread와 process 2가지로 나누어 숫자 야구 게임에서 숫자를 선택할 수 있는 방식의 차이점을 확인하고자 하였다. 최대 4명까지의 client가 server에 연결하여 메시지를 보내고 각 메시지를 받은 server가 전체 client에게 동일한 응답을 할 수 있게 하였다. 숫자 야구 게임은 thread 방식으로 숫자를 선택해 게임을 진행하였다. 추가적으로, thread 방식으로 숫자를 선택하는 방식을 process 방식으로도 구현하여 두 방식의 차이점에 대해서도 확인할 수 있었다.

Ⅰ. 서론

해당 구현물은 숫자 야구 게임을 TCP 통신하는 server-client 관계로 구현하였다. Server 한 개에 최대 4명의 client가 접속해 게임을 같이 진행한다. 숫자 야구 게임에서 숫자 3개를 선택하는 방식을 thread 방식과 process 방식, 이 두 가지 방식을 사용하였다. 이를 통해 동일한 코드 흐름을 thread로 수행하였을 때와 process로 수행하였을 때의 차이에 대해 파악하고자 하였다.

Ⅱ. 배경 지식

① TCP: TCP는 연결 지향형 프로토콜이다. TCP의 통신 과정을 server와 client측으로 나누어 살펴보자면, server측은 소켓을 생성하고 소켓을 주소와 바인딩한 후 client로부터 연결 요청을 받아 연결을 한다. 이후 메시지를 주고 받을 수 있으며, 소켓을 닫아 종료한다. Client 측은 소켓을 생성하고 server에 연결을 시도한다. 연결에 성공을 하면 메시지를 주고 받을 수 있고, 소켓을 닫아 종료하게 된다.

② Thread: 하나의 프로세스에서 실행되는 여러 흐름의 단위이다. 하나의 프로세스에 존재하는 스레드들은 메모리 공간을 공유하기 때문에 공유 자원을 처리하는데 주의를 해야한다. 또한, 여러 스레드가 동시에 실행되었을 때, 어떤 스레드가 먼저 실행되는지나 종료되는지를 예측할 수 없다. Thread는 pthread_create으로 생성을 할 수 있는데, 이 때 스레드가 실행할 함수, 함수로 보낼 인자를 결정할 수 있다. 또한, pthread_join을 통해 스레드가 종료될 때까지 대기 할 수 있다.

③ Process: 프로세스는 실행 중인 프로그램의 인스턴스이다. (실행 중이지 않는 프로세스도 존재하긴 하다.) Process는 fork를 통해 자식 프로세스를 생성할 수 있고, fork 실행 후 리턴된 값이 0이면 자식 프로세스, 0이 아닌 값이면 부모 프로세스에서 자식 프로세스의 pid을 리턴받는다. 자식 프로세스는 부모 프로세스의 주소 공간을 복사해서 만들지만, 독립적인 메모리 공간을 갖기 때문에 변수를 독립적으로 사용한다. 또한, fork를 통해 자식 프로세스를 생성하면 어떤 프로세스(부모 프로세스, 자식 프로세스)가 먼저 실행되고, 종료될지 예측할 수 없다.

⑤ Mutex: 스레드나 프로세스가 공유 자원에 접근하는 것을 통제하는 역할을 한다. Lock을 사용해서 누군가가 사용하고 있음을 알리고 다른 스레드나 프로세스가 사용하지 않게 한다. 그리고 사용을 마치면 unlock을 통해 다른 스레드/프로세스가 사용할 수 있게 한다.

⑥ Waitpid:

```
#include <sys/wait.h>
pid_t waitpid(pid_t pid, int *status, int options);
```

자식 프로세스를 기다릴 때 사용하는 함수이다. 해당 함수를 통해 자식 프로세스의 종료 상태를 확인할 수 있다. 첫 번째 인자(pid)는 기다리고자하는 자식 프로세스의 PID이다. -1인 경우는 모든 자식 프로세스를 기다린다. 두 번째 인자(status)는 자식 프로세스의 종료 상태를 저장하는 포인터이다. 마지막 세 번째 인자(options)는 옵션으로, 프로젝트에서는 사용하지 않았다. 또한, 상태에 대한 정보를 얻기 위해 자식이 종료되었는지를 확인할 수 있는 WIFEXITED와 WEXITSTATUS를 통해 종료된 자식 프로세스의 리턴 값을 확인할 수 있다.

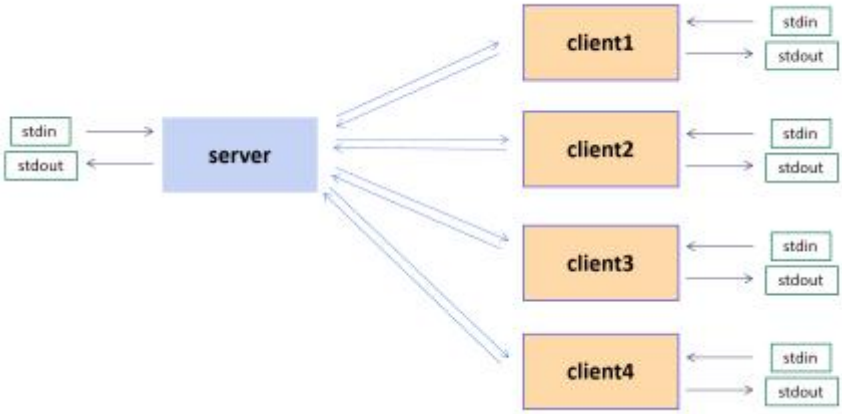
⑦ 라이브러리 설명

<stdio.h>: 표준 입출력 관련 라이브러리(printf, scanf, exit)
<stdlib.h>: 난수 발생, 문자열 변환, 메모리 동적 할당, 종료 등과 관련된 라이브러리(srand, rand, atoi, malloc, exit 등)
<time.h>: 날짜와 시간 관련 라이브러리 (time)
<unistd.h>: 유닉스/리눅스 시스템 호출 라이브러리(write, read, close, usleep, fork 등)
<string.h>: 문자열과 메모리 처리 라이브러리(strlen, strcmp, strcpy, memset 등)
<arpa/inet.h>: ip 주소 변환 관련 라이브러리(inet_addr, htons)
<sys/socket.h>: 소켓 관련 라이브러리(socket, connect, struct sockaddr_in, AF_INET, SOCK_STREAM)
<pthread.h>: 스레드 관련 라이브러리(pthread_create, pthread_join, pthread_exit)
<sys/wait.h>: 자식 프로세스 종료 관련 라이브러리.(wait, waitpid)

⑧ 개발 환경: Linux-Ubuntu 22.04.4 LTS

⑨ 사용된 프로토콜: TCP

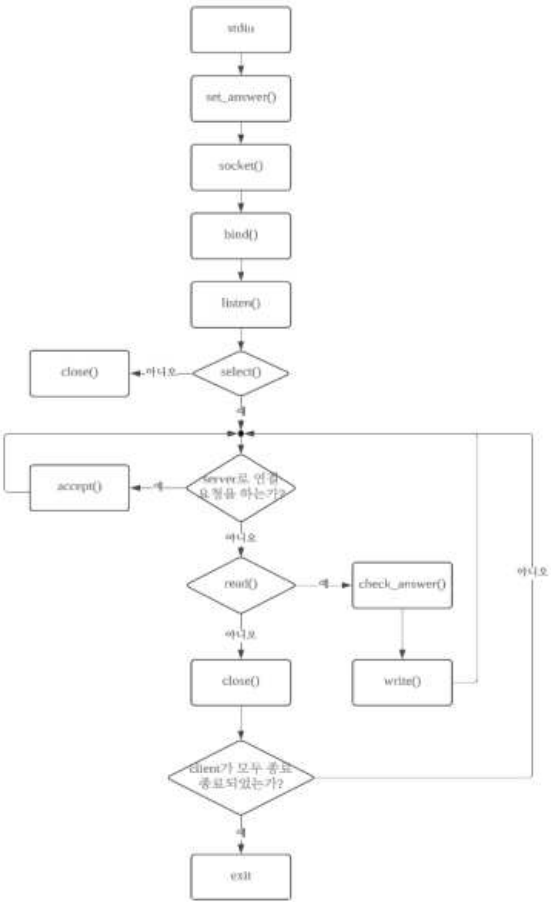
Ⅲ. 프로젝트 설명



<프로젝트 구현물에 대한 개념도>

*시스템 구성에 대한 도식화 및 모듈별 설명

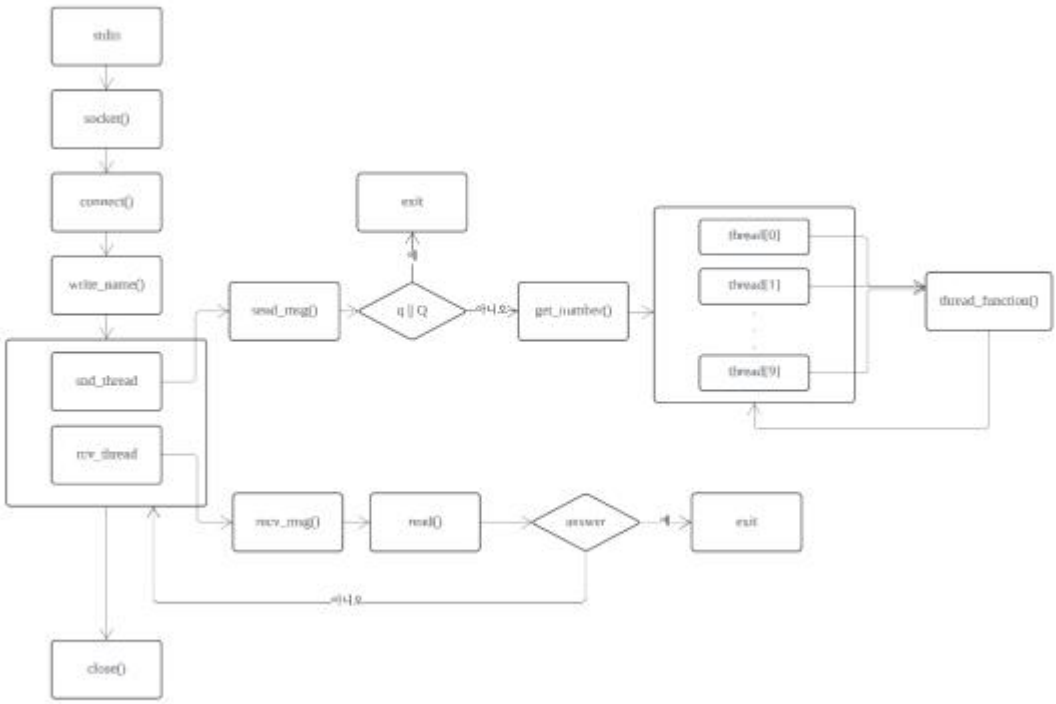
<Server.c>



<Server.c 도식화>

1) stdin: 포트 번호를 입력 받음
2) set_answer: 숫자 야구 게임의 정답을 설정
3) socket(): 서버 소켓 생성
4) bind(): 서버 주소를 서버 소켓에 바인딩
5) listen(): client의 연결 요청 대기
6) select(): 수신된 데이터가 있는지 확인(오류 시 소켓 닫기)
7) accept(): server로 연결 요청을 하는 client를 수락
8) read(): client가 보낸 메시지 읽기
9) check_answer(): client가 선택한 숫자 3개가 정답인지 확인
10) write(): client에게 답변 전달
11) close(): 소켓 닫기

<Client.c>

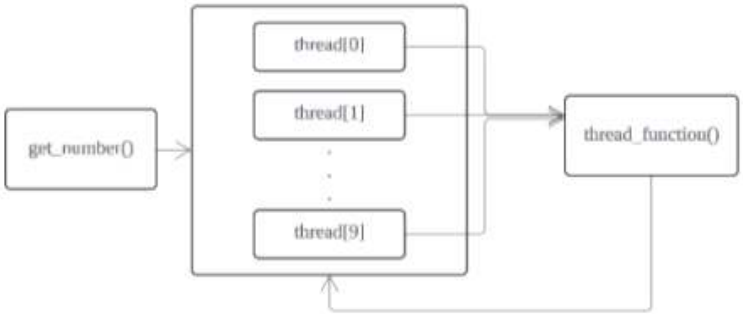


<Client.c 도식화>

1) socket(): 소켓 생성
2) connect(): 서버와 연결 시도
3) write_name(): 사용자 이름 server에 전송
4) snd_thread: 메시지를 서버에 보내는 스레드. send_msg() 함수 실행
5) rcv_thread: 서버로부터 메시지를 받는 스레드. recv_msg() 함수 실행
6) send_msg(): 사용자로부터 받은 메세지가 q(Q)이면 종료. 아닐시 get_number 실행
7) get_number(): thread 0번부터 9번까지 각각의 인자(0~9)를 통해 thread_function 실행
8) thread_function(): 각 스레드의 실행 순서에 따라 전역 변수에 스레드 번호 저장(스레드

순서를 조금 더 다양화하기 위해 usleep 함수 사용)
9) recv_msg(): 메시지를 서버에게 받음. 만약 정답을 맞췄을 시 종료 아닐 시 다시 정답 맞추기 시작
9) close(): 소켓 닫기

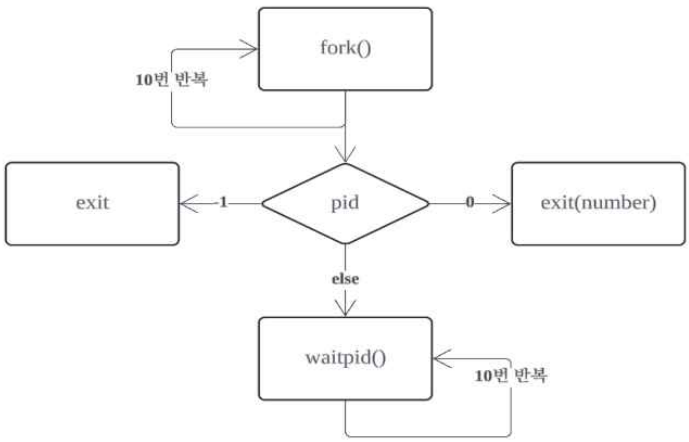
<threadtime.c>



<threadtime.c 도식화>

1) get_number(): thread 0번부터 9번까지 각각의 인자(0~9)를 통해 thread_function 실행
2) thread_function(): 각 스레드의 실행 순서에 따라 전역 변수에 스레드 번호 저장(스레드 순서를 조금 더 다양화하기 위해 usleep 함수 사용)

<processtime.c>



<processtime.c 도식화>

1) fork()를 10번 호출하여 자식 프로세스 10개를 생성한다. 이 때 각 프로세스가 들고 들어가는 number 값은 0~9이다. 이후 이 값을 exit할 때 리턴한다.
2) waitpid()도 10번 호출하여 자식 프로세스가 모두 종료되는 것을 기다린다. 이때 WEXITSTATUS(status)로 리턴 값을 받아 먼저 리턴 된 3개의 숫자를 저장한다.

***대표적인 모듈 및 함수의 실행 흐름**

<Client 코드 일부(get_number 함수, thread_function 함수)의 pseudo code>

```
void get_number()
{
        pthread_t threads[THREAD_COUNT];
        srand(time(NULL));

        initialize mutex;
        select_num[0]=select_num[1]=select_num[2]=0;
        entry=0;

        for i from 0 to 9:
                create threads[i];
                thread_function(i);

        for i from 0 to 9:
                pthread_join(threads[i], NULL);

        destroy mutex;
        return;
}

<thread_function>
void* thread_function(arg) {
    usleep((rand()%6000));
    mutex lock;
    if(entry<3) select_num[entry++]=arg;
    mutex unlock;

    return NULL;
}
```

Ⅳ. 결과 분석

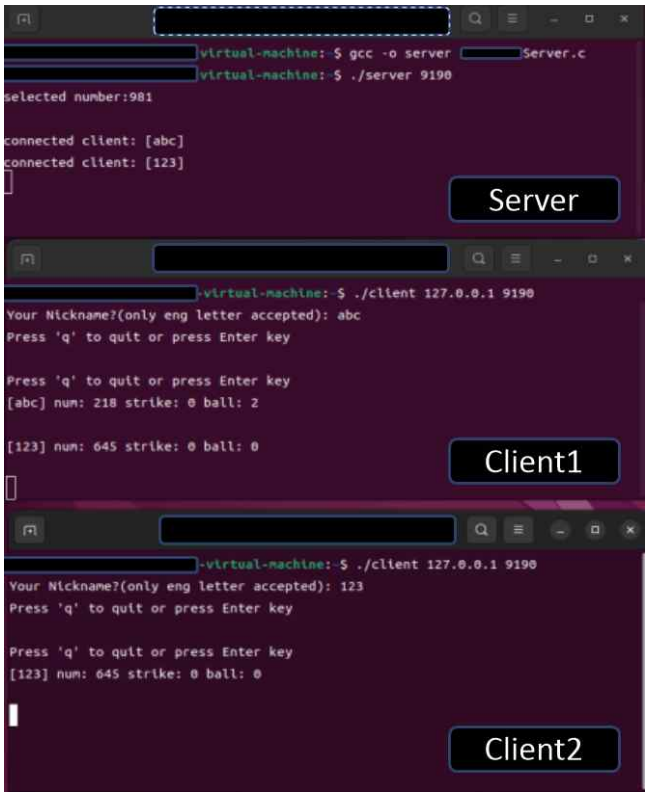
실행 방법:

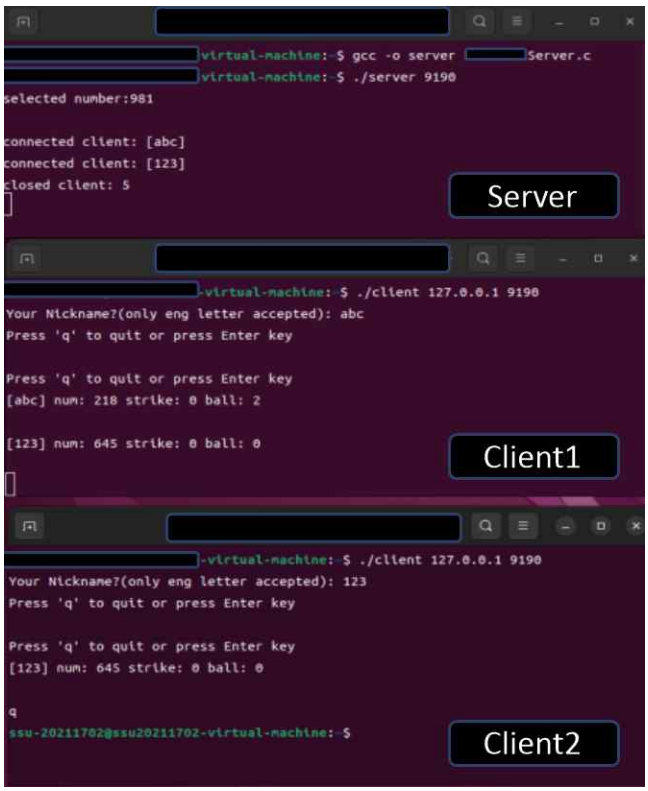
① server 먼저 실행. 파일 실행 시 포트 번호 입력

② client 실행. IP 주소와 포트 번호 입력

③ client에서 본인 이름(닉네임) 입력

④ client에서 엔터를 눌러 숫자 생성 및 서버에게 전달. q를 누르면 종료





⑤ 정답을 맞춘 client는 종료

⑥ 모든 client가 종료되면 server도 종료

복잡도, 지연 시간, 자원 소모량:

프로그램 실행할 때 발생하는 결과가 매번 달라지지만, client 2명이서 게임을 진행했을 때 실행 결과 중 평균적인 값의 결과를 살펴보았다. 또한, Server.c의 복잡도는 메시지 한 번 보내는 한 턴을 기준으로 계산하였다.

<Server.c>

① 복잡도: O(n)

```
real    0m48.135s
user    0m0.004s
sys     0m0.143s
```

② 실제 경과 시간: 0m 48.135s

③ 사용자 모드에서 소비된 CPU 시간: 0m 0.004s

④ 커널 모드에서 소비된 CPU 시간: 0m 0.143s

<Client.c>

① 복잡도: O(n)

[첫번째 client]

```
real    0m36.306s
user    0m0.044s
sys     0m2.088s
```

② 실제 경과 시간: 0m 36.306s

③ 사용자 모드에서 소비된 CPU 시간: 0m 0.044s

④ 커널 모드에서 소비된 CPU 시간: 0m 2.088s

[두 번째 client]

```
real    0m45.479s
user    0m0.003s
sys     0m0.105s
```

실제 경과 시간: 0m 45.479s

사용자 모드에서 소비된 CPU 시간: 0m 0.003s

커널 모드에서 소비된 CPU 시간: 0m 0.105s

<processtime.c>
매번 다른 값이 발생하지만 평균에 가까운 결과를 제시하였다.
① 시간 복잡도: O(1)

```
real    0m0.391s
user    0m0.003s
sys     0m0.005s
```

② 실제 경과 시간: 0m 0.391s
③ 사용자 모드에서 소비된 CPU 시간: 0m 0.003s
④ 커널 모드에서 소비된 CPU 시간: 0m 0.005s

<threadtime.c>
매번 다른 값이 발생하지만 평균에 가까운 결과를 제시하였다.
① 시간 복잡도: O(1)

```
real    0m0.863s
user    0m0.004s
sys     0m0.001s
```

② 실제 경과 시간: 0m 0.863s
③ 사용자 모드에서 소비된 CPU 시간: 0m 0.004s
④ 커널 모드에서 소비된 CPU 시간: 0m 0.001s

Ⅴ. 한계점 및 개선 가능성

<processtime.c>에서 프로세스가 종료되고 waitpid로 리턴 값을 받아 숫자 저장 배열에 저장될 때, 프로세스가 종료되는 순서대로 입력이 되지 않았다. 따라서, 중간에 지연시간이 발생한다는 것을 확인할 수 있고 먼저 종료되는 프로세스가 가지고 있는 값이 반영이 제대로 안 될 수 있다. 또한, <threadtime.c>에서 thread는 thread가 실행하는 함수(thread_function)에서 숫자 선택을 하는데, <processtime.c>에서는 프로세스는 종료가 된 이후에 값을 입력하게 된다. 따라서, thread와 process를 사용하였을 때 시간의 차이를 정확하게 파악하였다고 할 수 없다. 이를 해결하기 위해 process에서는 자식 프로세스가 실행이 되는 도중에 숫자를 출력하게 하는 것이 적절한 선택일 것으로 보인다.

Ⅵ. 결론

이번 프로젝트를 통해서 TCP 통신을 하는 방법에 대해 다시 한 번 확인하고 직접 구현해 볼 수 있었다. 각 client와의 연결이 지속되는 상황에서 client가 종료되었을 때에 대한 server의 처리 방안을 생각해 볼 수 있고, 모든 client끼리 소통할 수 있는 방법 역시도 함께 구현해 볼 수 있었다.

Thread와 process의 차이점이 분명하게 보였던 부분은 공유 자원의 유무였다. Thread 방식으로는 공유 자원을 이용하여 각 thread가 실행됨에 따라 정보를 공유 자원(전역 변수)에 저장할 수 있었는데, process에서는 변수를 독립적으로 사용하기 때문에 waitpid로 status를 받아 확인하는 방법을 선택하였다. 추가적으로, thread 생성이 process 생성보다 빠르다고 수업 시간에 들은 경험이 있는데, 이번 프로젝트에서는 thread의 결과가 더 느리게 나온 것을 확인할 수 있었다. *정확하게 왜 이런 결과가 나왔는지는 추후에 더 공부하면서 파악해보고자 한다.

*CPU 코어 개수, context switch 비용, mutex 등이 원인인 것으로 보인다.